

实验11. 聚类与数据可视化

实验目的

1. 实现kMeans聚类算法，并将其应用于把客户群体划分（可供营销团队参考并相应地制定营销策略）
2. 体验使用pandas和seaborn类库对数据进行统计和可视化

实验任务数据

mall_Customers.csv - 超市购物中心200名客户的一些基本数据，包括如下5个属性：

- CustomerID：客户ID
- Gender：性别
- Age：年龄
- Annual Income：年收入（单位：千元）
- Spending Score (1-100)：消费指数

【注】.csv文件是使用逗号和回车区分数据列和行的文本数据集，能够被文本编辑器、Excel和各类数据处理类库访问的一种文件格式。

1. 数据统计及可视化

1.1 使用Pandas数据读取及属性显示

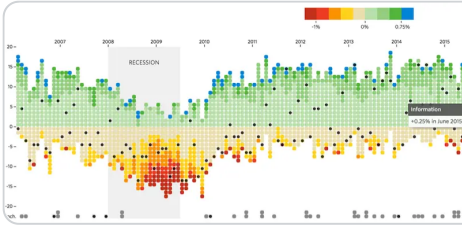
使用pandas.read_csv()读取商场消费者数据，并展示数据集的相关信息。

```
1  import pandas as pd
2
3  df = pd.read_csv('./mall_Customers.csv') #返回对象为pandas.DataFrame类型的，后面
      几个都是DataFrame的方法
4  df.head()      # 用来显示数据集的前5个样本数据
5  df.shape      # 显示数据集的尺寸
6  df.describe()  #显示各个特征的统计信息，包括样本数、均值、标准差、最小值、1/4分位
      数、1/2分位数、3/4分位数和最大值。
7  df.dtypes      # 展示各个字段的类型信息
8  df.isnull().sum() # 该数据集没有取值为null的属性值，可用此函数找空值。
```

1.2 使用seaborn和matplotlib数据可视化

<https://seaborn.pydata.org/>

seaborn: statistical data visualization — seaborn 0.13.2 documentation



知乎 https://www.zhihu.com/tardis/bd/art/81553421?source_id=1001

数据可视化，Seaborn画图原来这么好看

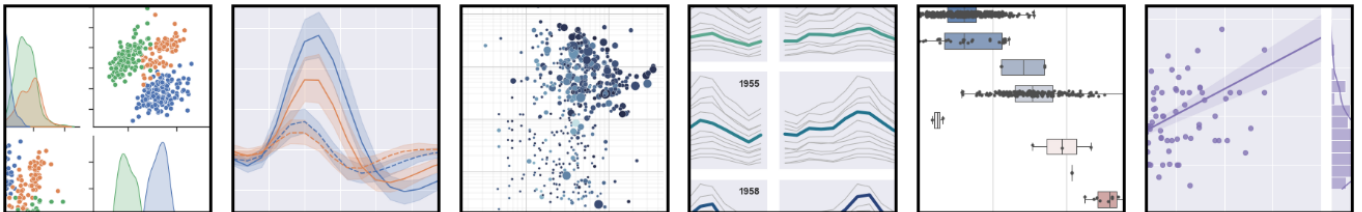
数据可视化，Seaborn画图原来这么好看 1927 赞同 38 评论 8119 收藏 matplotlib是python最常见的绘图包，强大之处不言而喻。然而在数据科学领域，可视化库...



Installing Gallery Tutorial API Releases Citing FAQ



seaborn: statistical data visualization



Seaborn is a Python data visualization library based on [matplotlib](#). It provides a high-level interface for drawing attractive and informative statistical graphics.

For a brief introduction to the ideas behind the library, you can read the [introductory notes](#) or the [paper](#). Visit the [installation page](#) to see how you can download the package and get started with it. You can browse the [example gallery](#) to see some of the things that you can do with seaborn, and then check out the [tutorials](#) or [API reference](#) to find out how.

To see the code or report a bug, please visit the [GitHub repository](#). General support questions are most at home on [stackoverflow](#), which has a dedicated channel for seaborn.

Contents

[Installing](#)
[Gallery](#)
[Tutorial](#)
[API](#)
[Releases](#)
[Citing](#)
[FAQ](#)

Features

- [New](#) Objects: [API](#) | [Tutorial](#)
- Relational plots: [API](#) | [Tutorial](#)
- Distribution plots: [API](#) | [Tutorial](#)
- Categorical plots: [API](#) | [Tutorial](#)
- Regression plots: [API](#) | [Tutorial](#)
- Multi-plot grids: [API](#) | [Tutorial](#)
- Figure theming: [API](#) | [Tutorial](#)
- Color palettes: [API](#) | [Tutorial](#)

1.2.1 统计直方图1：展示主要属性的数量分布

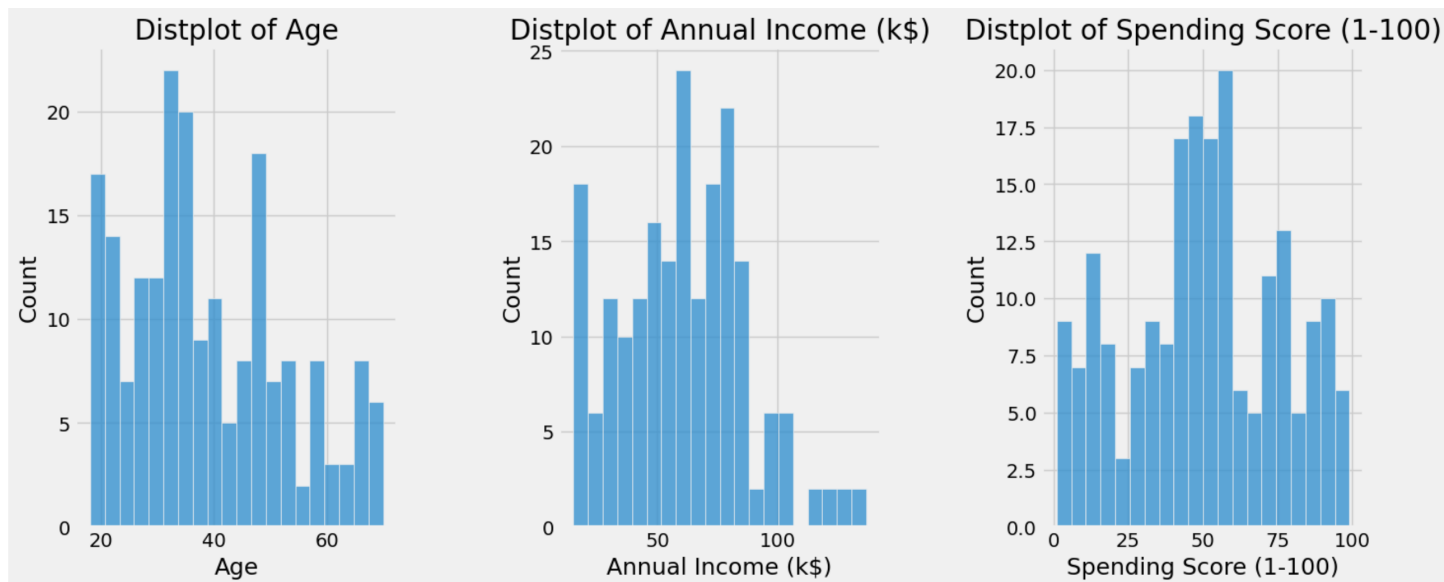
绘制数据的统计直方图，使用下方代码对其中三个主要属性的统计直方图，包含年龄、收入、消费指数。此处涉及matplotlib的FiveThirtyEight模式、多子图绘制，以及seaborn的分布直方图显示函数 `histplot()`

```
1  import matplotlib.pyplot as plt
2  import seaborn as sns
3
4  plt.style.use('fivethirtyeight')
5  plt.figure(1, figsize = (15, 6))
6  n = 0
7  for x in ['Age', 'Annual Income (k$)', 'Spending Score (1-100)']:
8      n += 1
9      plt.subplot(1, 3, n)
10     plt.subplots_adjust(hspace = 0.5, wspace = 0.5)
```

```

11     sns.histplot(df[x] , bins = 20)
12     plt.title('Distplot of {}'.format(x))
13     plt.show()

```



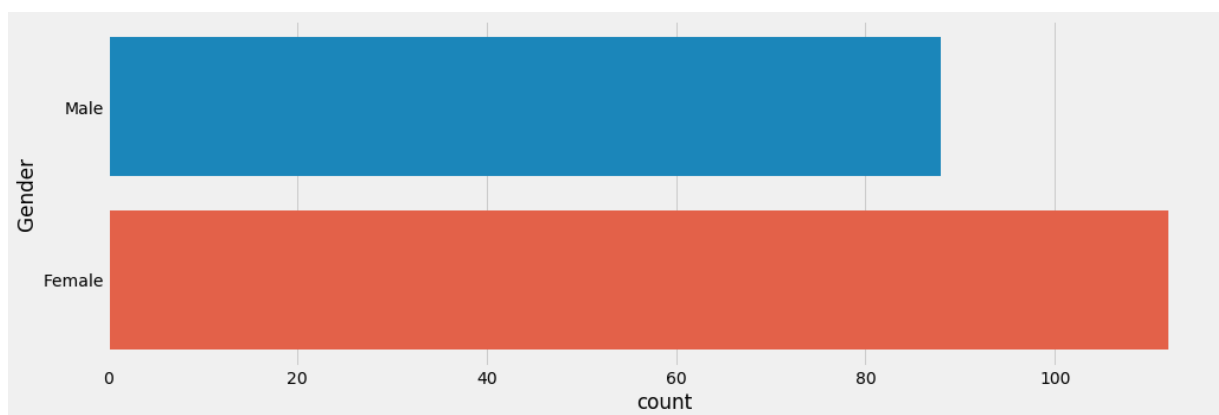
1.2.2 统计直方图2：展示男、女客户数量的分布

使用seaborn的计数直方图绘制男、女样本数量分布。

```

1  plt.figure(1 , figsize = (15 , 5))
2  sns.countplot(y = 'Gender' , data = df)
3  plt.show()

```



1.2.3 展示属性间的相关性

通过多子图绘制、散点图和回归模型（seaborn的regplot()）展现主要属性（年龄、年收入、消费指数）间的线性相关程度。

```

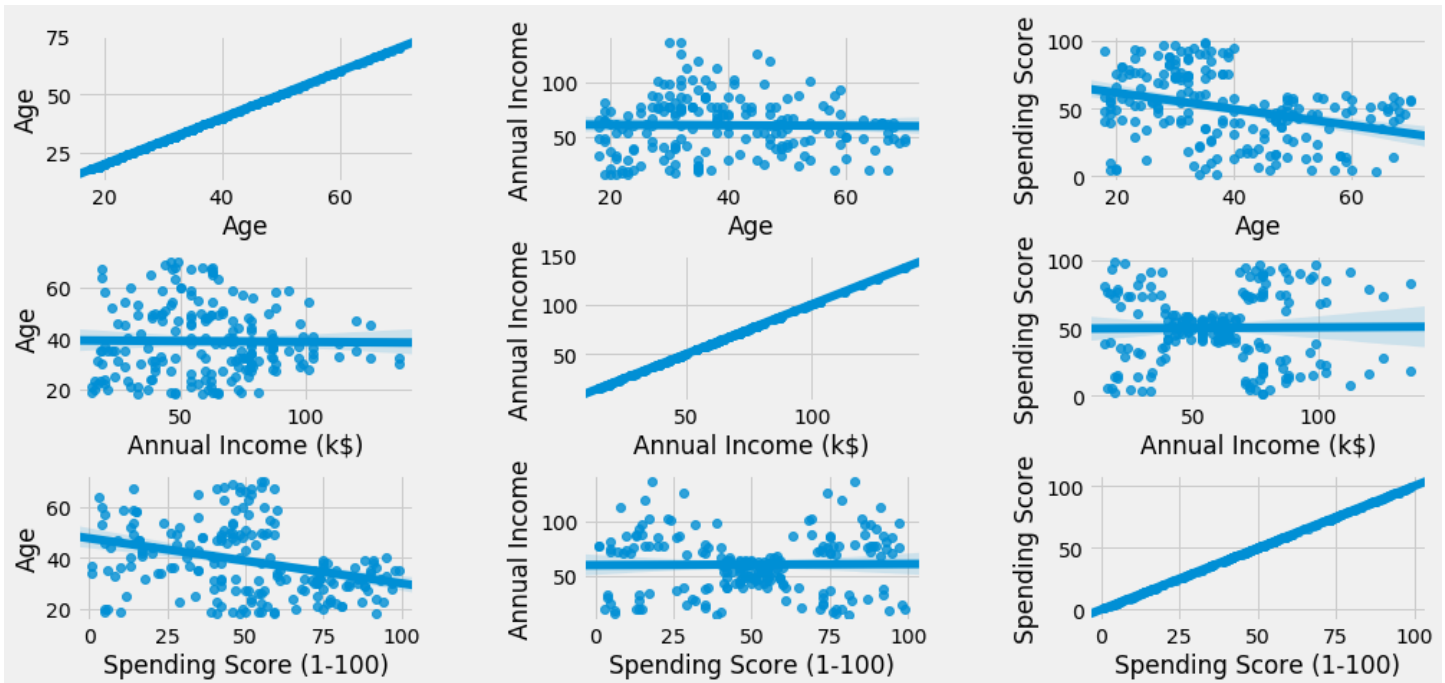
1  plt.figure(1 , figsize = (15 , 7))
2  n = 0

```

```

3  for x in ['Age' , 'Annual Income (k$)' , 'Spending Score (1-100)']:
4      for y in ['Age' , 'Annual Income (k$)' , 'Spending Score (1-100)']:
5          n += 1
6          plt.subplot(3 , 3 , n)
7          plt.subplots_adjust(hspace = 0.5 , wspace = 0.5)
8          sns.regplot(x = x , y = y , data = df)
9          plt.ylabel(y.split()[0]+' '+y.split()[1] if len(y.split()) > 1 else y
10         )
11     plt.show()

```



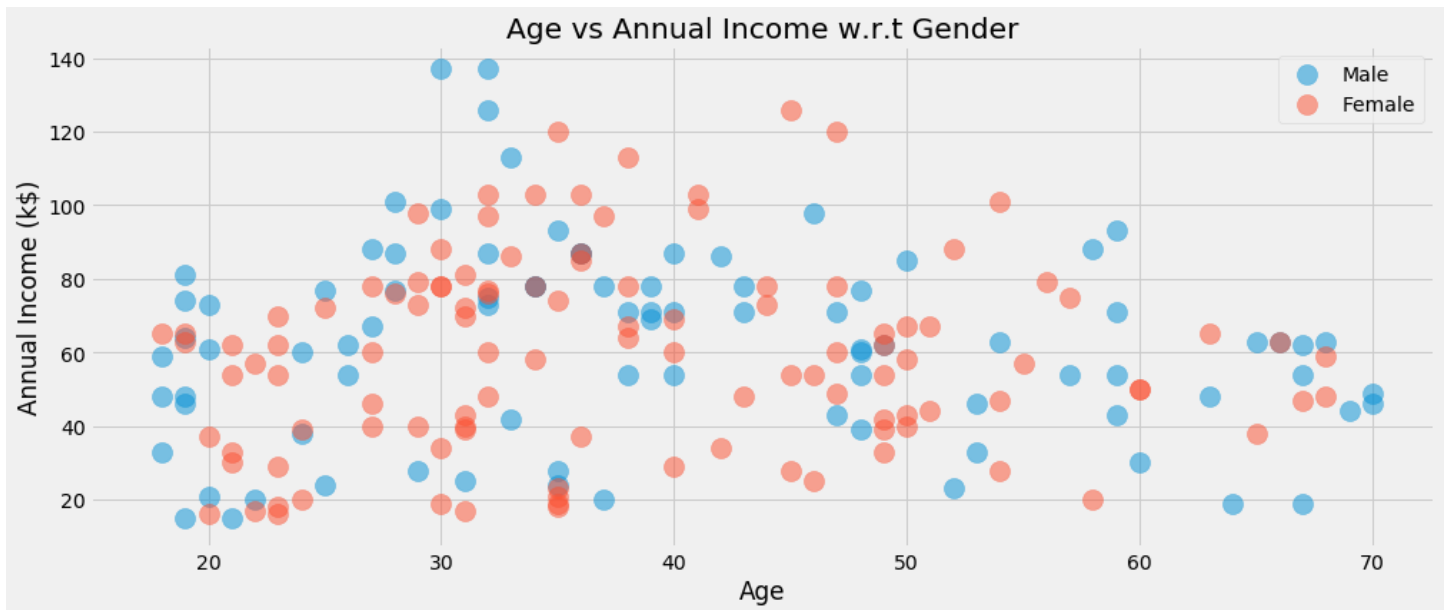
1.2.4 指定属性显示分类数据

在上述选定两属性进行样本点显示基础上，可以使用不同风格显示不同类别的数据。比如在“年龄”和“年收入”坐标系下按性别显示相关用户。

```

1  plt.figure(1 , figsize = (15 , 6))
2  for gender in ['Male' , 'Female']:
3      plt.scatter(x = 'Age' , y = 'Annual Income (k$)' ,
4                  data = df[df['Gender'] == gender] ,
5                  s = 200 , alpha = 0.5 , label = gender)
6  plt.xlabel('Age')
7  plt.ylabel('Annual Income (k$)')
8  plt.title('Age vs Annual Income w.r.t Gender')
9  plt.legend()
10 plt.show()

```



1.2.5 使用小提琴风格的数据分布可视化

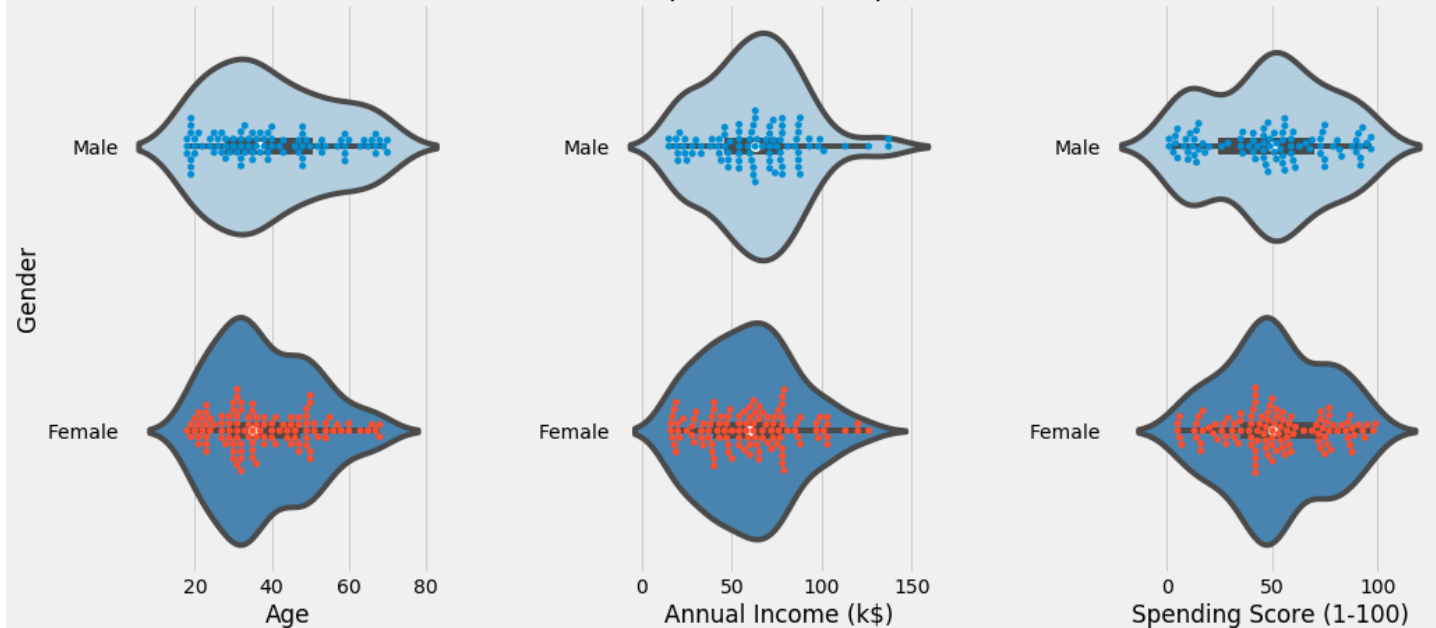
数据分布也可以指定部分数据（如gender = male or female）下其他属性的取值分布。涉及到seaborn的violinplot()和swarmplot()

```

1  plt.figure(1 , figsize = (15 , 7))
2  n = 0
3
4  for cols in ['Age' , 'Annual Income (k$)' , 'Spending Score (1-100)']:
5      n += 1
6      plt.subplot(1 , 3 , n)
7      plt.subplots_adjust(hspace = 0.5 , wspace = 0.5)
8      sns.violinplot(x = cols , y = 'Gender' , data = df, palette='Blues')
9      sns.swarmplot(x = cols , y = 'Gender' , data = df)
10     plt.ylabel('Gender' if n == 1 else '')
11     plt.title('Boxplots & Swarmplots' if n == 2 else '')
12 plt.show()

```

Boxplots & Swarmplots



2. k均值聚类 (k-means clustering)

k-means聚类算法是应用最为广泛最容易被理解的聚类算法，它通过数据相似程度将 m 个样本分成 k 个簇，期望达到的目标是：每个样本和同簇样本更为接近，而非同簇样本相距更远。具体的算法在下图中给出。

Algorithm 1: k-means (k 均值) 聚类

Input: 无标注数据集 D ，聚簇个数 k ，最大迭代次数 T

Output: k 个簇: $C_1, C_2, \dots, C_k$, $C_1 \cup C_2 \cup \dots \cup C_k = D$

```

1  随机初始化  $k$  个簇中心  $\mu_1, \mu_2, \dots, \mu_k$ 
2  for  $t = 1, \dots, T$  do
3      for  $i = 1, \dots, m$  do
4           $c^{(i)} = \arg \min_{j \in \{1, 2, \dots, k\}} \text{dist}(\mathbf{x}^{(i)}, \mu_j)$ 
5      end
6      for  $j = 1, \dots, k$  do
7           $\mu_j = \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}$ 
8      end
9      if  $k$  个簇的均值  $\mu_j$  均不发生变化 then
10         break
11     end
12 end
```

2.1 实现k-means聚类算法

请根据上面的算法和脚手架代码完成该算法，主要用于计算每个样本到k个均值间的距离。

```
1  import numpy as np
2
3  def kmeans(X, K, max_iters=100):
4      # 随机初始化质心
5      centroids = X[np.random.choice(range(X.shape[0]), K, replace=False)]
6
7      for _ in range(max_iters):
8          # 一个临时的距离矩阵distances, 用于存储每个数据点与每个质心之间的距离, 规模为
            X.shape[0]*K。
9          #distances第i行第j列表示第i个数据点到第j个质心的距离, 计算两个点x1,x2之间的距
            离距离可以使用np.linalg.norm (x1-x2)
10         '''
11
12         code here
13
14         '''
15
16         # 分配每个数据点到最近的质心
17         labels = np.argmin(distances, axis=1)
18
19         # 更新质心位置
20         new_means = np.array([X[labels == k].mean(axis=0) for k in range(K)])
21
22         # 如果质心位置不再改变, 停止迭代
23         if np.all(means == new_means):
24             break
25
26         means = new_means
27
28         # 计算各样本到所属簇中心的距离和
29         # inertia = np.sum(''distances'')
30         return labels, means
31
32 # 示例用法
33 X = np.array([[1, 2], [1, 4], [1, 0], [4, 2], [4, 4], [4, 0]])
34 K = 2
35 labels, means = kmeans(X, K)
36 print("聚类结果: ", labels)
37 print("质心位置: ", means)
```

2.2 使用k-means对客户数据进行聚类

从mall_consumers数据集中选取“年龄”和“消费指数”两列，分别使用刚实现的k-means()和sklearn.cluster.KMeans()对上述子集进行聚类，并针对不同K取值，展现出聚类评价指标的变化。

其中，inertia（惯性）是指数据点与其最近聚类中心之间的距离的加权总和。它值越小越好，表示样本在类间的分布越集中。这也可看作k-means算法的优化目标。即：

$$L(kMeans, D) = \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} dist(\mathbf{x}, \boldsymbol{\mu}_i)$$

2.2.1 使用刚实现的k-means算法进行聚类

```
1  '''Age and spending Score'''
2  X1 = df[['Age' , 'Spending Score (1-100)']].iloc[: , :].values
3  labels= []
4  means = []
5  labels, means = kmeans(X1, K, 300)
6  print("聚类结果: ", labels)
7  print("质心位置: ", centroids)
```

Cell输出结果(结果可能有所不同):

 聚类结果：[010
1010101010000011000001001000100100000
0100100100100110000001010100010000000
1011101001011010101010101010101010101
0101010101010101010101010101010101010
101010101010101]

质心位置：[[1.75 1.5]
[4. 3.]]

2.2.2 使用sklearn.cluster.KMeans算法进行聚类

关于函数参数的说明:

init--质心的初始化方式, 'random'

n init--用不同的质心初始化值运行算法的次数，最终解是在inertia意义下选出的最优结果

tol--迭代停止

random_state--用于初始化质心的生成器 (generator)

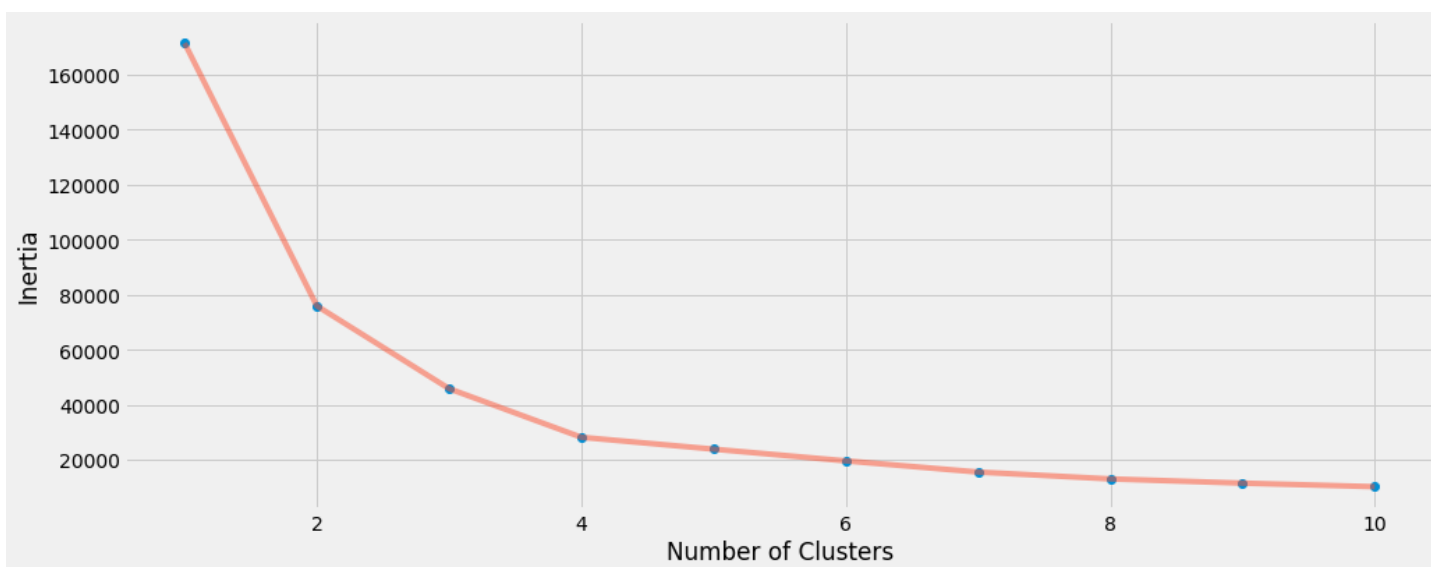
algorithm-- “auto”，“full” or “elkan” 三种选择。“full”就是我们传统的K-Means算法，“elkan”是我们原理篇讲的elkan K-Means算法。默认的”auto”则会根据数据值是否是稀疏的，来决定如何选择”full”和“elkan”

```
1 from sklearn.cluster import KMeans
2 '''Age and spending Score'''
3 X1 = df[['Age' , 'Spending Score (1-100)']].iloc[: , :].values
4 inertia = []
5 for n in range(1 , 11):
6     algorithm = (KMeans(n_clusters = n ,init='k-means++', n_init = 10
7         ,max_iter=300,
8         tol=0.0001, random_state= 111 , algorithm='elkan') )
9     algorithm.fit(X1)
10    inertia.append(algorithm.inertia_)
```

2.2.3 可视化k-means算法的优化目标与参数k之间的关系

观察10次聚类的inertias，并以如下折线图进行统计。其中，inertias值越小越好，越小表示样本在类间的分布越集中。

```
1 plt.figure(1 , figsize = (15 ,6))
2 plt.plot(np.arange(1 , 11) , inertia , 'o')
3 plt.plot(np.arange(1 , 11) , inertia , '-' , alpha = 0.5)
4 plt.xlabel('Number of Clusters') , plt.ylabel('Inertia')
5 plt.show()
```



2.2.4 可视化聚类结果——年龄和消费指数间的关系

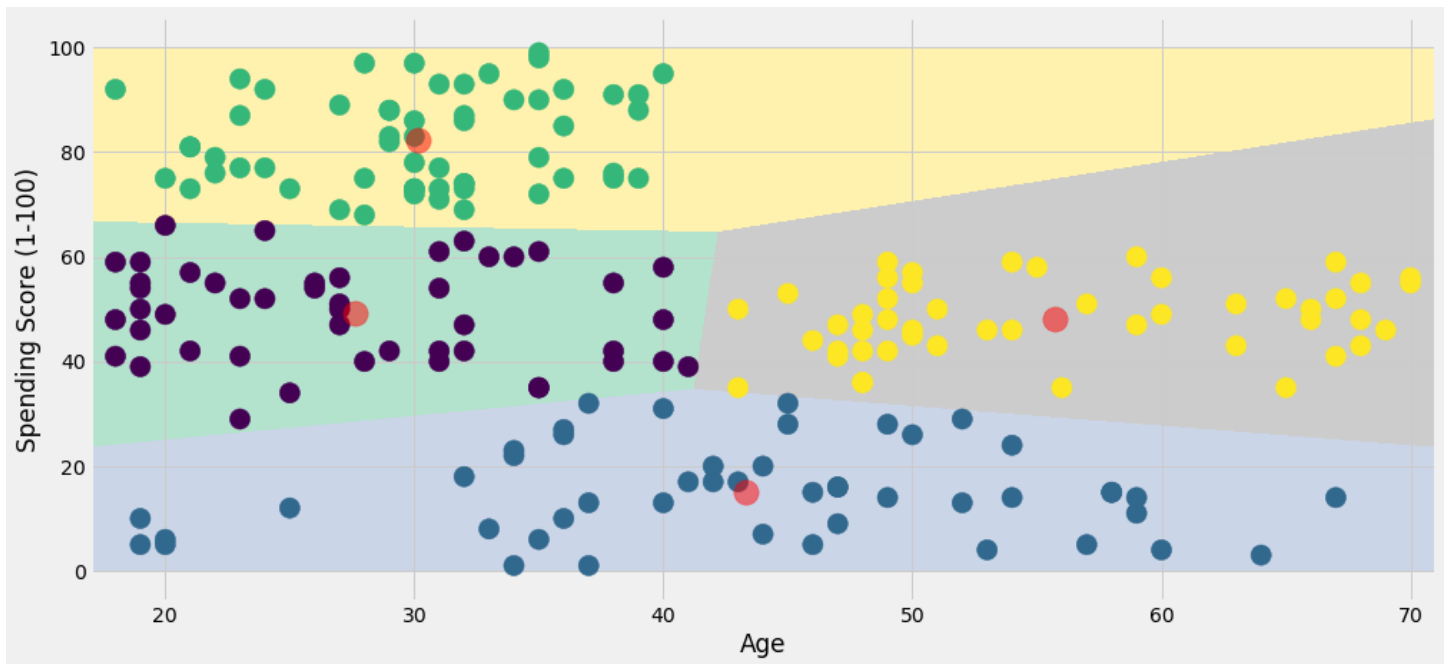
该图可以进一步反映上一节中训练得到的线性回归模型无法很好地拟合训练数据，具体体现为训练误差（error_train）和验证误差（error_val）都比较高。尤其是作为基本条件的训练误差很高（high bias），而此种情况反映的是模型过于简单导致线性模型无法拟合非线性数据集。需要增强模型的描述能力，即将线性回归扩展为可以拟合非线性数据的多项式回归。

```
1  algorithm = (KMeans(n_clusters = 4 ,init='k-means++', n_init = 10
    ,max_iter=300,
2                                tol=0.0001,  random_state= 111 , algorithm='elkan') )
3  algorithm.fit(X1)
4  labels1 = algorithm.labels_
5  centroids1 = algorithm.cluster_centers_
```

把4个聚类中心的聚类结果在下图进行展示。横坐标是年龄，纵坐标是消费指数，4个红点为4个聚类中心，4块不同颜色区域就是4个不同的用户群体。

```
1  h = 0.02
2  x_min, x_max = X1[:, 0].min() - 1, X1[:, 0].max() + 1
3  y_min, y_max = X1[:, 1].min() - 1, X1[:, 1].max() + 1
4  xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
5  Z = algorithm.predict(np.c_[xx.ravel(), yy.ravel()])
6
7  plt.figure(1 , figsize = (15 , 7) )
8  plt.clf()
9  Z = Z.reshape(xx.shape)
10 plt.imshow(Z , interpolation='nearest',
11             extent=(xx.min(), xx.max(), yy.min(), yy.max()),
12             cmap = plt.cm.Pastel2, aspect = 'auto', origin='lower')
13
14 plt.scatter( x = 'Age' ,y = 'Spending Score (1-100)' , data = df , c =
    labels1 ,
15             s = 200 )
16 plt.scatter(x = centroids1[:, 0] , y = centroids1[:, 1] , s = 300 , c =
    'red' , alpha = 0.5)
17 plt.ylabel('Spending Score (1-100)') , plt.xlabel('Age')
18 plt.show()
```

运行结果显示如下：



2.2.5 聚类及可视化2——五类用户群里的年收入和消费指数

请仿照上一节的代码，完成一下工作

1. 从数据集中选择“年收入”和“消费指数”两列对用户进行聚类（使用 `sklearn.cluster.KMeans()`），在簇中心个数 k 的选择上依然使用1到10；
2. 观察10次聚类的代价函数值 `inertias`，并使用折线图进行展示；
3. 设定聚簇个数 $k=5$ ，再次使用 `sklearn.cluster.KMeans()` 进行聚类，并仿照上一节对数据结果进行可视化。

